

OS Exercise 2

Operating Systems Reichman University

12.05.2025

Acknowledgments

We would like to thank the Hebrew University for the original idea and examples.

Submission Notes:

- **Submission Deadline:** Via Moodle by 9.06.2025. **No late submissions will be accepted!**
- **Collaboration:** This assignment is to be done in pairs (i.e., two students per team).
- **Submission Format:** Submit a single ZIP file containing all the requested files.
- **README:** Include a `README.txt` file at the root of the ZIP. This file must contain the submitting students' information. Each line should follow this format:

First name, Last name, Student ID

- **Working Environment:** This assignment requires the use of an Ubuntu OS with x86 64 bit architecture, be sure to work with the appropriate virtual machine/container/device.
- **Compilation Environment:** The checker uses `gcc13` on Ubuntu 24.04 LTS with the C17 standard (Default in GCC13).
- **Compilation and Execution:** Ensure your code compiles without errors and runs as expected.
- **Testing:** Write your own unit tests to validate your implementation by creating a `testuthreads.c` file containing a `main` function that tests your functions. **Do not submit `testuthreads.c`.** Make sure your submitted files **do not contain** a `main` function.
- **Covered Material:** This assignment covers topics from Presentation 3 and Recitation 3.
- **Header File Modification - You can't Modify the header file (including the structs)**
- **Dynamic Memory Allocation:** - Dynamic memory allocation (i.e `malloc`) is not allowed (and is not needed).

Introduction

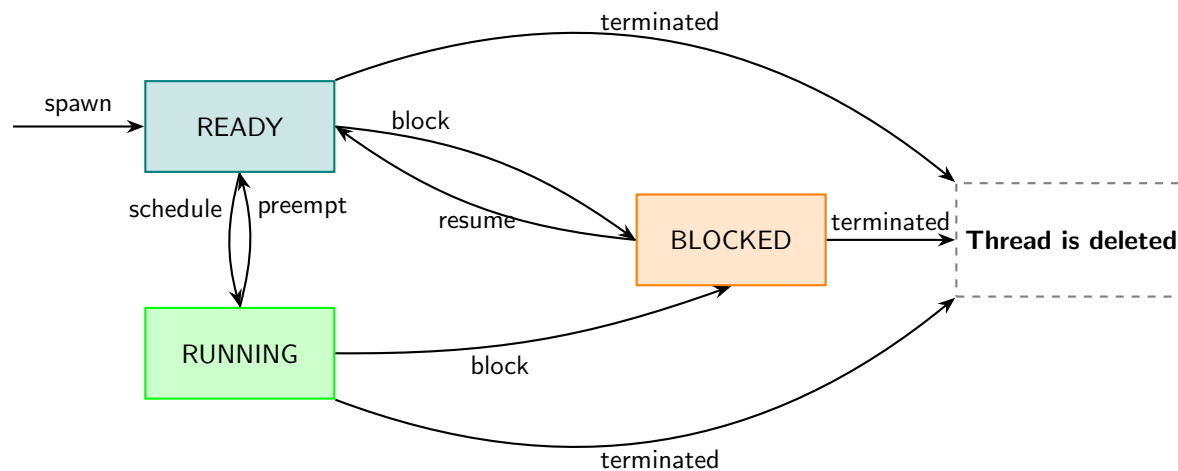
In this assignment, you will build a static library (compiled code without `main`) that creates and manages user-level threads. Your library's header is defined at `uthreads.h`. Your primary task is to implement all the functions specified in the `uthreads` API according to their documentation. You may also need to design and implement internal helper functions and data structures. There is no restriction on how many or what form these internal components take.

The Threads

Every program begins with a default main thread, which has a user-level thread ID of 0. All other threads must be explicitly created by the user-level library. Each thread is assigned a unique, non-negative integer as its ID. When creating a new user-thread (i.e. `uthread_spawn`), you must assign it the smallest non-negative integer that is not currently in use (for example, if thread 1 terminates and a new thread is created, it should receive the ID 1). The library must support at most `MAX_THREAD_NUM` threads at any time, including the main thread.

Every thread will always be in one of the following states: `RUNNING`, `BLOCKED`, or `READY`.

Thread State Diagram



Example two user threads with two separate stacks.

```
1 /*
2  * Simplified cooperative threads demo using sigsetjmp/siglongjmp.
3  * This example is written for 64-bit Intel architectures only.
4  * Two threads are set up with separate stacks, and they switch control
5  * by saving and restoring execution contexts.
6  */
7
8 #include <stdio.h>
9 #include <setjmp.h>
10 #include <signal.h>
11 #include <unistd.h>
12 #include <stdbool.h>
13
14 // Define a type for addresses.
15 typedef unsigned long address_t;
16
17 // These indices refer to the positions in the jmp_buf where the stack pointer
18 // and program counter are stored.
19 #define JB_SP 6
20 #define JB_PC 7
21
22 // This function adjusts an address for use with sigsetjmp/siglongjmp.
23 // The inline assembly is used as a black box to "translate" the address.
24 address_t translate_address(address_t addr) {
25     address_t ret;
26     asm volatile("xor    %%fs:0x30, %0\n"
27                 "rol     $0x11, %0\n"
```

```

27         : "=g" (ret)
28         : "0" (addr));
29     return ret;
30 }
31
32 // Constants for time and stack size.
33 #define SECOND 1000000        // 1 second in microseconds.
34 #define STACK_SIZE 4096      // Each thread gets a 4096-byte stack.
35
36 // Define a type for thread functions.
37 typedef void (*thread_entry_point)(void);
38
39 // Allocate separate stacks for the two threads.
40 char stack0[STACK_SIZE];
41 char stack1[STACK_SIZE];
42
43 // Create jump buffers to store the execution contexts for each thread.
44 sigjmp_buf env[2];
45 // This variable keeps track of the currently running thread (0 or 1).
46 int current_thread = -1;
47
48 // Function to switch to a specific thread using siglongjmp.
49 void jump_to_thread(int tid) {
50     current_thread = tid;
51     siglongjmp(env[tid], 1);
52 }
53
54 // Yield function: save the current thread's state and switch to the other thread.
55 void yield(void) {
56     // Save the current execution context. sigsetjmp returns 0 when saving.
57     int ret_val = sigsetjmp(env[current_thread], 1);
58     // When coming back from a siglongjmp, sigsetjmp returns a nonzero value.
59     if (ret_val == 0) {
60         // Switch to the other thread (if current is 0, switch to 1; if 1, switch
61         // to 0).
62         jump_to_thread(1 - current_thread);
63     }
64     // If ret_val is nonzero, the thread is resuming execution.
65 }
66
67 // The first thread: counts and yields every 3 iterations.
68 void thread0(void) {
69     int counter = 0;
70     while (1) {
71         counter++;
72         printf("Thread 0: counter = %d\n", counter);
73         // Yield control after every 3 iterations.
74         if (counter % 3 == 0) {
75             printf("Thread 0 yielding...\n");
76             yield();
77         }
78         usleep(SECOND); // Sleep for 1 second.
79     }
80 }
81
82 // The second thread: counts and yields every 5 iterations.
83 void thread1(void) {
84     int counter = 0;
85     while (1) {

```

```

85     counter++;
86     printf("Thread 1: counter = %d\n", counter);
87     // Yield control after every 5 iterations.
88     if (counter % 5 == 0) {
89         printf("Thread 1 yielding...\n");
90         yield();
91     }
92     usleep(SECOND); // Sleep for 1 second.
93 }
94 }
95
96 // Set up a thread by initializing its jump buffer to use a new stack and start
97 // at a given function.
98 void setup_thread(int tid, char *stack, thread_entry_point entry_point) {
99     // Calculate the initial stack pointer (start at the top of the stack).
100     address_t sp = (address_t)stack + STACK_SIZE - sizeof(address_t);
101     // Get the function pointer for the thread's starting function.
102     address_t pc = (address_t)entry_point;
103     // Save the current context into the jump buffer.
104     sigsetjmp(env[tid], 1);
105     // Manually set the stack pointer and program counter in the jump buffer.
106     env[tid]->__jmpbuf[JB_SP] = translate_address(sp);
107     env[tid]->__jmpbuf[JB_PC] = translate_address(pc);
108     // Clear the saved signal mask.
109     sigemptyset(&env[tid]->__saved_mask);
110 }
111
112 // Initialize both threads.
113 void setup(void) {
114     setup_thread(0, stack0, thread0);
115     setup_thread(1, stack1, thread1);
116 }
117
118 // Main entry point: set up threads and start execution.
119 int main(void) {
120     setup();
121     // Begin execution with thread 0.
122     jump_to_thread(0);
123     return 0;
124 }

```

Listing 1: Simplified cooperative threads demo using sigsetjmp/siglongjmp (64-bit Intel architectures)

Scheduler

Your library must include a scheduler that manages thread execution using the Round-Robin (RR) scheduling algorithm. This algorithm works by allocating a fixed time slice, called a *quantum*, to each thread when it is in the RUNNING state.

Time

The process running time is measured by the Virtual Timer. Virtual time refers to the amount of CPU time a process actually uses, rather than the real (or wall clock) time that passes in the system. This distinction is crucial for understanding how threads are scheduled and managed in this assignment.

- **Virtual Time:**

- Measures the CPU time consumed by the process.
- Advances only when the process (or one of its threads) is actively running.
- Used to allocate and monitor a thread's quantum—i.e., the fixed time slice a thread receives when it is in the RUNNING state.
- Typically implemented using the `setitimer` system call with the `ITIMER_VIRTUAL` option.

- **Wall Clock Time:**

- Represents the actual elapsed time in the real world.
- Includes all time intervals, whether the process is running, waiting, or blocked.
- Not used for scheduling in this assignment, as we focus solely on the process's active (virtual) running time.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <sys/time.h>
4 #include <unistd.h>
5
6 // Signal handler for SIGVTALRM
7 void timer_handler(int signum) {
8     // This function is called each time the virtual timer expires
9     printf("Virtual timer expired\n");
10 }
11
12 int main(void) {
13     struct sigaction sa;
14     struct itimerval timer;
15
16     // Set up the SIGVTALRM handler using sigaction
17     sa.sa_handler = timer_handler;    // Specify our signal handler
18     sigemptyset(&sa.sa_mask);        // No signals blocked during the handler
19     sa.sa_flags = 0;                 // No special flags
20
21     // Register the SIGVTALRM handler
22     if (sigaction(SIGVTALRM, &sa, NULL) == -1) {
23         perror("sigaction");
24         return 1;
25     }
26
27     // Configure the virtual timer:
28     // The timer will first expire after 1 second, and then every 1 second
29     // thereafter.
30     timer.it_value.tv_sec = 1;        // Initial expiration in seconds
31     timer.it_value.tv_usec = 0;       // Initial expiration in microseconds
32     timer.it_interval.tv_sec = 1;     // Subsequent intervals in seconds
33     timer.it_interval.tv_usec = 0;    // Subsequent intervals in microseconds
34
35     // Start the virtual timer (ITIMER_VIRTUAL counts CPU time used by the
36     // process)
37     if (setitimer(ITIMER_VIRTUAL, &timer, NULL) == -1) {
38         perror("setitimer");
39         return 1;
40     }
41
42     // Busy loop to consume CPU time so that the virtual timer counts down.
43     while (1) {
44         // Perform some CPU-bound task; this loop will trigger timer signals.

```

```

43     // Note: In a real application, replace this with useful work.
44 }
45
46 return 0;
47 }

```

Listing 2: Example of using `sigaction` with a virtual timer (`ITIMER_VIRTUAL`)

Algorithm

The Round-Robin scheduling policy should operate as follows:

1. **Quantum Allocation:** Whenever a thread enters the `RUNNING` state, it is allocated a fixed number of microseconds to execute (the quantum).
2. **Preemption Conditions:** The currently running thread is preempted if:
 - (a) Its allocated quantum expires.
 - (b) It voluntarily blocks itself (for example, if it is waiting for an event or resource).
 - (c) It terminates (either by its own action or via a call from another thread).
3. **Handling Preemption:** When a thread is preempted:
 - (a) If the preemption is due to quantum expiration, move the thread to the end of the `READY` queue.
 - (b) Immediately schedule the next thread from the front of the `READY` queue, switching it to the `RUNNING` state.
4. **Queue Management:** Every time a thread transitions into the `READY` state from any other state, it should be added to the end of the `READY` queue.
5. **Partial Quantum Usage:** If a thread does not consume its entire quantum (for instance, if it blocks before the quantum expires), the scheduler should immediately start the next thread as though the previous thread had fully used its time slice.

Library Functions

The API for your thread library is defined in the header file `uthread.h`. When these functions are called, they will cause state transitions as shown in the state diagram. A thread may invoke a function on its own ID (affecting its state) or on another thread's ID (affecting that thread's state).

Notes

1. You must implement and manage a data structure (such as a queue or list) to maintain the threads in the `READY` state. You are free to use any additional data structures that help achieve the required functionality.
2. If a thread is both blocked and sleeping, it should remain inactive until both conditions are resolved: its sleep duration has expired and another thread resumes it.
3. The main thread (i.e. `tid == 0`) operates using the same stack, program counter, and registers that were active when `uthread_init` was called. Therefore, you do not need to allocate a separate stack or manually set its SP and PC. Nonetheless, it should be managed just like any other thread during context switches.
4. **You must protect your "critical" state update sections with signal masking** to prevent a user-level context switch (that is caused by the timer signal) in order to prevent state corruption. If any of your masking operations fail, then print to `stderr` "system error: masking failed" and exit the process with an exit code of 1.

5. You **are not allowed to use dynamic allocations (i.e malloc)**, You may use a static array to manage the thread's TCBs and the thread Stacks.

Simplifying Assumptions

You may assume the following:

1. Every thread will call `uthread_terminate` before returning, whether it terminates itself or is terminated by another thread.
2. The allocated stack space for each spawned thread is sufficient and will not be exceeded during execution.
3. Neither the main thread nor any thread created via the `uthreads` library will manipulate timer signals (specifically `SIGVTALRM`) by sending, masking, or setting interval timers for them.

Error Messages

Your library should output error messages exclusively to `stderr`. No output should be sent to `stdout`. Use the following formats:

System Call Failure: When a system call fails (e.g., memory allocation), print a single line in the following format:

```
system error: text
```

Here, `text` should briefly describe the error. After printing, immediately call `exit(1)`.

Thread Library Failure: When a function in your thread library encounters an error (e.g., invalid input), print a single line in the following format:

```
thread library error: text
```

Again, `text` should provide a brief description of the error. Then, return the appropriate error value from the function.

The Library:

```
1 #ifndef _UTHREADS_H
2 #define _UTHREADS_H
3
4 #include <stdlib.h>
5 #include <signal.h>
6 #include <setjmp.h>
7 #include <stdbool.h>
8 #include <sys/time.h>
9 #include <unistd.h>
10 #include <string.h>
11 #include <stdio.h>
12
13 /* ===== */
14 /*                      Static Constants                      */
15 /* ===== */
16
17 /** Maximum number of threads (including the main thread). */
18 #define MAX_THREAD_NUM 100
19
20 /** Stack size per thread (in bytes). */
21 #define STACK_SIZE 4096
22
23 /**
```

```

24  * @brief Function pointer type for a thread's entry point.
25  *
26  * Each thread's entry function must take no arguments and return void.
27  */
28 typedef void (*thread_entry_point)(void);
29
30 /* ===== */
31 /*                               Internal Data Structures                               */
32 /* ===== */
33
34 /**
35  * @brief Enumeration of possible thread states.
36  */
37 typedef enum {
38     THREAD_UNUSED = 0, /**< Slot is unused. */
39     THREAD_READY,      /**< Thread is ready to run. */
40     THREAD_RUNNING,     /**< Thread is currently executing. */
41     THREAD_BLOCKED,     /**< Thread is blocked (explicitly or sleeping). */
42     THREAD_TERMINATED  /**< Thread has finished execution (internal use only). */
43 } thread_state_t;
44
45 /**
46  * @brief Thread Control Block (TCB)
47  *
48  * Each thread (except for the main thread) has its own allocated stack and
49  * context.
50  * The TCB stores all metadata required for managing the thread.
51  */
52 typedef struct {
53     int tid; /**< Unique thread identifier. */
54     thread_state_t state; /**< Current thread state. */
55     sigjmp_buf env; /**< Jump buffer for context switching using
56         sigsetjmp/siglongjmp. */
57     int quantum; /**< Count of quantum this thread has executed.
58         */
59     int sleep_until; /**< Global quantum count until which the thread
60         should sleep (0 if not sleeping). */
61     thread_entry_point entry; /**< Entry point function for the thread. */
62 } thread_t;
63
64 /* ===== */
65 /*                               External Interface                               */
66 /* ===== */
67
68 /**
69  * @brief Initializes the user-level thread library.
70  *
71  * This function must be called before any other thread library function.
72  * It sets up internal data structures, initializes the main thread (tid == 0) as
73  * running,
74  * and configures the timer for quantum management. The main thread uses the
75  * process's regular stack.
76  *
77  * @param quantum_usecs Length of a quantum in microseconds (must be positive).
78  * @return 0 on success; -1 on error (e.g., if quantum_usecs is non-positive).
79  */
80 int uthread_init(int quantum_usecs);
81
82 /**

```



```

77  * @brief Creates a new thread.
78  *
79  * Allocates a new TCB and separate stack for the thread (using a char array).
80  * The thread is added to the end of the READY queue.
81  * Calling this function with a NULL entry_point or exceeding MAX_THREAD_NUM is
    an error.
82  *
83  * @param entry_point Pointer to the thread's entry function (must not be NULL).
84  * @return On success, returns the new thread's ID; on failure, returns -1.
85  */
86  int uthread_spawn(thread_entry_point entry_point);
87
88  /**
89  * @brief Terminates a thread.
90  *
91  * Terminates the thread with the specified tid and releases all resources
    allocated for it.
92  * The thread is removed from all scheduling structures. If no thread with the
    given tid exists,
93  * it is considered an error. Terminating the main thread (tid == 0) will
    terminate the entire process
94  * (after releasing allocated resources).
95  *
96  * @param tid Thread ID to terminate.
97  * @return 0 on success; -1 on error. (Note: if a thread terminates itself or if
    the main thread terminates,
98  * the function does not return.)
99  */
100  int uthread_terminate(int tid);
101
102  /**
103  * @brief Blocks a thread.
104  *
105  * Moves the thread with the given tid into the BLOCKED state.
106  * A blocked thread may later be resumed using uthread_resume.
107  * It is an error to block a thread that does not exist or to block the main
    thread (tid == 0).
108  * Blocking a thread that is already BLOCKED is a no-op.
109  *
110  * @param tid Thread ID to block.
111  * @return 0 on success; -1 on error.
112  */
113  int uthread_block(int tid);
114
115  /**
116  * @brief Resumes a blocked thread.
117  *
118  * Moves a thread from the BLOCKED state to the READY state.
119  * If the thread is already in RUNNING or READY state, this call has no effect.
120  * It is an error if no thread with the given tid exists.
121  *
122  * @param tid Thread ID to resume.
123  * @return 0 on success; -1 on error.
124  */
125  int uthread_resume(int tid);
126
127  /**
128  * @brief Puts the running thread to sleep.
129  *

```

```

130 * Blocks the currently running thread for a specified number of quanta.
131 * The current quantum is not counted; sleeping begins with the next quantum.
132 * After the sleep period expires, the thread is moved to the end of the READY
    queue.
133 * It is an error for the main thread (tid == 0) to call this function.
134 *
135 * @param num_quantums Number of quanta to sleep.
136 * @return 0 on success; -1 on error.
137 */
138 int uthread_sleep(int num_quantums);
139
140 /**
141  * @brief Returns the calling thread's ID.
142  *
143  * @return The thread ID of the calling thread.
144  */
145 int uthread_get_tid();
146
147 /**
148  * @brief Returns the total number of quanta since the library was initialized.
149  *
150  * The count starts at 1 immediately after uthread_init. Every new quantum,
    regardless of cause,
151  * increments this counter.
152  *
153  * @return Total number of quanta.
154  */
155 int uthread_get_total_quantums();
156
157 /**
158  * @brief Returns the number of quanta the thread with the specified tid has
    run.
159  *
160  * For a thread in the RUNNING state, the current quantum is included.
161  * An error is returned if no thread with the given tid exists.
162  *
163  * @param tid Thread ID.
164  * @return Number of quanta for the specified thread; -1 on error.
165  */
166 int uthread_get_quantums(int tid);
167
168 /* ===== */
169 /*           Internal Helper Functions and Structures           */
170 /* ===== */
171 /*
172  * The following declarations are intended for internal use by the thread library.
173  * They provide guidance on how to structure your implementation using
    sigsetjmp/siglongjmp
174  * and manually managed stacks.
175  */
176
177 /**
178  * @brief Scheduler: Selects the next thread to run.
179  *
180  * This function examines the READY queue and selects the next thread for
    execution.
181  * It handles state transitions and triggers a context switch.
182  */
183 void schedule_next(void);

```

```

184
185 /**
186  * @brief Context switch helper.
187  *
188  * Uses sigsetjmp and siglongjmp to save the current thread's context and restore
189  * the context of the next thread.
190  *
191  * @param current Pointer to the current thread's TCB.
192  * @param next Pointer to the next thread's TCB.
193  */
194 void context_switch(thread_t *current, thread_t *next);
195
196 /**
197  * @brief Timer signal handler.
198  *
199  * Registered as the handler for timer signals, this function updates global
200  * quantum counters
201  * and initiates a scheduling decision when a quantum expires.
202  *
203  * @param signum The signal number (e.g., SIGVTALRM).
204  */
205 void timer_handler(int signum);
206
207 /**
208  * @brief Initializes a thread's jump buffer.
209  *
210  * Sets up the thread's jump buffer for context switching by manually assigning
211  * the stack pointer
212  * and program counter using sigsetjmp/siglongjmp techniques. You may need to
213  * perform architecture-specific
214  * address translation (see provided reference implementation) when initializing
215  * the context.
216  *
217  * @param tid Thread ID.
218  * @param stack Pointer to the thread's allocated stack (a char array).
219  * @param entry_point Pointer to the thread's entry function.
220  */
221 void setup_thread(int tid, char *stack, thread_entry_point entry_point);
222
223 #endif /* _UTHREADS_H */

```

Listing 3: User-level thread library header file (uthreads.h)

Background Reading and Resources

To gain a complete understanding of the relevant system calls, please consult the following man pages:

- `setitimer` (2)
- `getitimer` (2)
- `sigaction` (2)
- `sigsetjmp` (3)
- `siglongjmp` (3)
- `sigprocmask` (2)
- `sigemptyset`, `sigaddset`, `sigdelset`, `sigfillset`, `sigismember` (3)

- sigpending (2)
- sigwait (3)

Example of using sigaction with sigint.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <signal.h>
4 #include <unistd.h>
5
6 // Signal handler for SIGINT
7 void signal_handler(int signum) {
8     printf("Received signal %d (SIGINT). Exiting gracefully...\n", signum);
9     exit(0);
10 }
11
12 int main(void) {
13     struct sigaction sa;
14
15     // Set up the sigaction structure to specify the signal handler
16     sa.sa_handler = signal_handler; // Set the signal handler function
17     sigemptyset(&sa.sa_mask);      // No additional signals blocked during
18                                     // execution of the handler
19     sa.sa_flags = 0;                // No special flags
20
21     // Register the signal handler for SIGINT
22     if (sigaction(SIGINT, &sa, NULL) == -1) {
23         perror("sigaction");
24         exit(1);
25     }
26
27     printf("Program is running. Press Ctrl+C to trigger SIGINT.\n");
28
29     // Main loop: the program will continue running until it receives SIGINT
30     while (1) {
31         sleep(1);
32     }
33
34     return 0;
35 }

```

Listing 4: Example of using sigaction to handle SIGINT in a general case

Final Notes

- **Read the Instructions Carefully:** Ensure that you adhere to all restrictions and requirements.
- **Testing:** Test your code thoroughly with various inputs to ensure correctness, especially under concurrent execution.
 - **Also test without a debugger** since debuggers may affect synchronization behavior and hide bugs.
 - **Debugging:** Start with a small number of threads (e.g., one, then two, then five) and increase gradually.
 - Use debug printouts to help identify issues, but **make sure to remove or comment out** any debug messages before submission as they may affect grading.

- **Code Quality:** Write clean, well-commented code to help the graders understand your implementation. Comments will also help you understand what you are trying to achieve.
- **Unit Tests:** You are (more than) encouraged to use unit tests to validate your synchronization mechanisms.
- **Grading Reminder:** A happy grader is a merciful grader!

Good Luck!